

Java Concurrency: Threads, Communication, Locks, **volatile**, Executors & Coordination

1. Ways to Create a Thread

There are several approaches you should know.

1.1 Extend **Thread**

```
class PaymentThread extends Thread {  
  
    @Override  
    public void run() {  
        System.out.println("Processing payment");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        PaymentThread thread = new PaymentThread();  
        thread.start();  
    }  
}
```

Important

Call:

```
thread.start();
```

not:

```
thread.run();
```

start() creates/schedules a new thread.

run() is simply a normal method invocation when called directly.

```
start() → new execution thread  
run()   → current thread executes method
```

Why extending **Thread** is usually not preferred

Java supports single inheritance:

```
class PaymentThread extends Thread
```

so the class cannot extend another class.

It also mixes:

```
Task
+
Thread management
```

into one class.

2. Implement `Runnable`

Usually preferable when you specifically need to represent a task.

```
class PaymentTask implements Runnable {

    @Override
    public void run() {
        System.out.println("Processing payment");
    }

    Thread thread = new Thread(new PaymentTask());
    thread.start();
}
```

Or with lambda:

```
Thread thread = new Thread(() -> {
    System.out.println("Processing payment");
});

thread.start();
```

Advantage

`Runnable` represents:

what work should be done

while `Thread` represents:

the execution mechanism

This separation becomes even more useful with executors.

3. `Callable` — Return a Result

`Runnable` cannot directly return a result.

`Callable<T>` can.

```
Callable<Integer> task = () -> {  
    return 100;  
};
```

Usually submitted to an executor:

```
ExecutorService executor =  
    Executors.newFixedThreadPool(4);  
  
Future<Integer> future = executor.submit(task);  
  
Integer result = future.get();  
  
executor.shutdown();
```

`future.get()` waits until the task completes.

4. Modern Java: Virtual Threads

Modern Java also provides **virtual threads**, introduced as a permanent feature in Java 21.

Example:

```
Thread.startVirtualThread(() -> {  
    processPayment();  
});
```

Or:

```
try (ExecutorService executor =  
    Executors.newVirtualThreadPerTaskExecutor()) {  
  
    executor.submit(() -> processPayment());  
}
```

Conceptually:

```
Platform thread  
  ↓  
OS-backed execution resource  
  ↓  
More expensive  
  
Virtual thread  
  ↓  
Managed by JVM  
  ↓  
Very lightweight  
  ↓  
Suitable for large numbers of blocking tasks
```

For an interview focused on Java 8/Spring Boot, know traditional threads and executors thoroughly, but being aware of virtual threads is useful.

5. Thread Lifecycle

A simplified lifecycle:

```
NEW
↓
RUNNABLE
↓
RUNNING
↓
BLOCKED / WAITING / TIMED_WAITING
↓
RUNNABLE
↓
TERMINATED
```

Examples:

```
thread.start();
```

moves the thread from **NEW** toward execution.

```
Thread.sleep(1000);
```

causes:

```
TIMED_WAITING
```

Waiting for a monitor:

```
lock
```

can result in:

```
BLOCKED
```

Calling:

```
wait()
```

results in:

```
WAITING
```

6. Thread Communication

Suppose Thread A produces data and Thread B consumes it.

```
Thread A
  Producer
    |
    | data
    ↓
  Shared resource
    |
```

↓
Thread B
Consumer

The threads need to coordinate.

Common mechanisms:

- shared memory + synchronization
- `wait()` / `notify()`
- `BlockingQueue`
- `CountDownLatch`
- `CyclicBarrier`
- `Semaphore`
- `Future`
- `CompletableFuture`

Modern Java applications generally prefer **higher-level concurrency utilities** rather than implementing coordination manually with `wait/notify`.

7. Race Condition

A race condition happens when multiple threads access shared mutable state and the result depends on timing.

Example:

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
}
```

This looks like one operation:

```
count++
```

but conceptually:

```
read count  
add 1  
write count
```

Two threads can do:

```
Thread A: read 0  
Thread B: read 0  
  
Thread A: write 1  
Thread B: write 1
```

Expected:

2

Actual:

1

That's a race condition.

8. `synchronized`

The simplest way to protect critical sections.

```
public synchronized void increment() {  
    count++;  
}
```

Or:

```
public void increment() {  
    synchronized (this) {  
        count++;  
    }  
}
```

The idea:

```
Thread A  
  ↓  
acquire monitor  
  ↓  
critical section  
  ↓  
release monitor  
  
Thread B  
  ↓  
waits
```

Only one thread at a time can enter the synchronized region protected by the same monitor.

9. What Does `synchronized` Provide?

Two major things:

Mutual exclusion

Only one thread enters the critical section at a time.

Visibility

Changes made before releasing the monitor become visible to a thread that subsequently acquires the same monitor.

So `synchronized` is not merely:

“preventing two threads from entering.”

It also establishes a **happens-before relationship**.

10. Object Lock vs Class Lock

Instance synchronized method

```
public synchronized void process() {  
}
```

Locks:

```
this
```

So different objects can have different locks.

Static synchronized method

```
public static synchronized void process() {  
}
```

Locks the class's monitor:

```
MyClass.class
```

Therefore all instances are synchronized against the same class-level monitor for that method.

11. Explicit Locks — `Lock`

Java also provides:

```
java.util.concurrent.locks.Lock
```

Most commonly:

```
ReentrantLock
```

Example:

```
private final Lock lock = new ReentrantLock();  
  
public void process() {  
    lock.lock();  
  
    try {
```

```

        // critical section
        updatePayment();
    } finally {
        lock.unlock();
    }
}

```

The `finally` is extremely important.
Otherwise an exception could leave the lock acquired.

12. Why `ReentrantLock` ?

Compared with `synchronized` , it provides additional capabilities.
For example:

`tryLock()`

```

if (lock.tryLock()) {
    try {
        process();
    } finally {
        lock.unlock();
    }
} else {
    System.out.println("Could not acquire lock");
}

```

A thread doesn't necessarily have to wait forever.

Timed lock

```

if (lock.tryLock(2, TimeUnit.SECONDS)) {
    try {
        process();
    } finally {
        lock.unlock();
    }
}

```

Interruptible acquisition

```
lock.lockInterruptibly();
```

These capabilities are reasons to choose an explicit `Lock` in some situations.

13. `synchronized` vs `ReentrantLock`

<code>synchronized</code>	<code>ReentrantLock</code>
Language-level construct	API
Automatically releases lock	Must manually unlock

Simple	More flexible
No <code>tryLock()</code>	Supports <code>tryLock()</code>
No timed acquisition directly	Supports timed acquisition
Less code	More control

A good interview answer:

I would prefer `synchronized` when simple mutual exclusion is sufficient. I would consider `ReentrantLock` when I need features such as timed acquisition, `tryLock`, interruptible locking, or more explicit lock management.

14. ReadWriteLock

Sometimes you have:

Many readers
Few writers

A normal lock unnecessarily prevents readers from reading simultaneously.

`ReentrantReadWriteLock` provides:

Read Lock
Write Lock

Example:

```
private final ReentrantReadWriteLock rwLock =
    new ReentrantReadWriteLock();

public String readData() {

    rwLock.readLock().lock();

    try {
        return data;
    } finally {
        rwLock.readLock().unlock();
    }
}
```

Multiple readers can potentially proceed concurrently.

Writer:

```
rwLock.writeLock().lock();

try {
    data = newValue;
} finally {
    rwLock.writeLock().unlock();
}
```

A write requires exclusive access.

15. `volatile`

This is one of the most frequently misunderstood Java concurrency concepts.

Consider:

```
class Worker {  
  
    private boolean running = true;  
  
    void stop() {  
        running = false;  
    }  
  
    void work() {  
        while (running) {  
            // do work  
        }  
    }  
}
```

You might assume Thread A changing:

```
running = false;
```

means Thread B immediately sees it.

Java's concurrency model requires proper visibility guarantees.

Using:

```
private volatile boolean running = true;
```

establishes the required visibility semantics for this simple flag.

16. What Does `volatile` Guarantee?

Primarily:

Visibility

A write by one thread to a volatile variable becomes visible to subsequent reads by other threads according to volatile's happens-before rules.

It does **not** generally provide:

Atomicity for compound operations

This is the classic trap.

17. `volatile` Does NOT Make `count++` Safe

This is still unsafe:

```
private volatile int count;

count++;
```

Why?

```
read
+
increment
+
write
```

Two threads can still race.

Use:

```
AtomicInteger count = new AtomicInteger();

count.incrementAndGet();
```

or synchronization/locking where appropriate.

18. When Should You Use `volatile`?

Good use case:

```
private volatile boolean shutdown;
```

Thread A:

```
shutdown = true;
```

Thread B:

```
while (!shutdown) {
    performWork();
}
```

You need visibility of a simple state flag.

Other examples include some configuration/state indicators.

Don't use `volatile` as a replacement for locking when several operations must be treated atomically.

19. Atomic Classes

Java provides:

```
AtomicInteger
AtomicLong
AtomicBoolean
AtomicReference
```

Example:

```
AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet();
```

This solves the `count++` problem without explicitly using a monitor.

These classes typically use **CAS (Compare-And-Set / Compare-And-Swap)** based atomic operations.

Conceptually:

```
current value = X

"Change X to Y only if current value is still X"
```

If another thread changed it:

```
CAS fails
```

and the implementation can retry.

20. `volatile` vs `AtomicInteger`

<code>volatile</code>	<code>AtomicInteger</code>
Visibility	Visibility + atomic operations
Simple state	Counters/state transitions
<code>count++</code> unsafe	<code>incrementAndGet()</code> safe
Does not provide compound atomicity	Provides atomic operations

Good interview phrase:

`volatile` is mainly about visibility; atomic classes provide atomic read-modify-write operations as well.

21. `wait()` / `notify()` / `notifyAll()`

These are methods of `Object`.

They are used for thread coordination through an object's **monitor**.

Classic example:

```
Producer
↓
puts item
↓
shared queue
↓
Consumer
↓
takes item
```

Consumer may need to wait while queue is empty.

22. Basic `wait/notify` Example

```
class SharedResource {

    private boolean available = false;

    public synchronized void consume()
        throws InterruptedException {

        while (!available) {
            wait();
        }

        System.out.println("Consumed");
        available = false;
    }

    public synchronized void produce() {

        available = true;
        notifyAll();
    }
}
```

Important

`wait()` releases the object's monitor.

So:

```
Thread A
↓
acquires monitor
↓
wait()
↓
releases monitor
↓
Thread B can enter
```

This is a major distinction from:

```
Thread.sleep(...)
```

23. `wait()` vs `sleep()`

Very common interview question.

<code>wait()</code>	<code>sleep()</code>
<code>Object</code> method	<code>Thread</code> method
Used for coordination	Used for pausing execution
Releases monitor	Does not release monitor
Usually called inside synchronized context	No monitor requirement
Can be awakened by <code>notify/notifyAll</code> , <code>interrupt</code> , etc.	Wakes after timeout/interruption

Think:

```
sleep() → "Pause me"
wait() → "I cannot continue until some condition changes"
```

24. Why `while`, Not `if`, Around `wait()` ?

Use:

```
while (!condition) {
    wait();
}
```

not:

```
if (!condition) {
    wait();
}
```

Because after waking up, the condition should be **checked again**.

Reasons include:

- another thread may have changed the state
- multiple waiting threads may wake
- the condition may no longer hold

This is a classic interview point.

25. Why `notifyAll()` Is Often Safer Than `notify()`

```
notify();
```

wakes one waiting thread.

```
notifyAll();
```

wakes all waiting threads.

After waking, they compete to reacquire the monitor and re-check their conditions.

With multiple kinds of waiting conditions, blindly using `notify()` can potentially wake a thread that cannot make progress.

For custom coordination code, `notifyAll()` is often easier to reason about.

That said, higher-level utilities such as `BlockingQueue` are usually preferable to hand-written producer/consumer logic.

26. Executors — The Preferred Way to Manage Threads

Instead of doing:

```
new Thread(...).start();
new Thread(...).start();
new Thread(...).start();
```

applications usually use an:

```
ExecutorService
```

Why?

Because creating and destroying threads repeatedly is expensive and unmanaged.

Executor:

```
Task
↓
Executor
↓
Thread Pool
↓
Worker Thread
↓
Execute Task
```

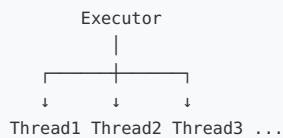
27. Fixed Thread Pool

```
ExecutorService executor =
    Executors.newFixedThreadPool(4);
```

Submit tasks:

```
executor.submit(() -> processPayment());
executor.submit(() -> processPayment());
executor.submit(() -> processPayment());
```

Conceptually:



At most the configured number of worker threads execute concurrently.

28. Why Thread Pools?

Suppose you receive:

10,000 requests

Creating:

10,000 threads

would be a terrible design in many ordinary server workloads.

Instead:

```
10,000 tasks
  ↓
bounded executor
  ↓
20 / 50 / 100 workers
```

Tasks wait in a queue when workers are busy.

Benefits:

- controls concurrency
- reuses threads
- reduces thread creation overhead
- provides task queuing
- gives operational control

29. `ExecutorService` Methods

`execute()`

```
executor.execute(() -> process());
```

For a `Runnable`.

`submit()`

```
Future<?> future =
    executor.submit(() -> process());
```

Can return a `Future` .

`shutdown()`

```
executor.shutdown();
```

Stops accepting new tasks but allows submitted tasks to finish.

`shutdownNow()`

Attempts to stop currently executing tasks, primarily through interruption.

It is not a guarantee that every task immediately stops.

30. `Future`

Suppose:

```
ExecutorService executor =  
    Executors.newFixedThreadPool(4);  
  
Future<Integer> future =  
    executor.submit(() -> calculateAmount());
```

You can later:

```
Integer amount = future.get();
```

But remember:

```
future.get()
```

can block.

You can also:

```
future.isDone();  
future.cancel(true);
```

31. `CompletableFuture`

For more sophisticated asynchronous composition:

```
CompletableFuture
```

Example:

```
CompletableFuture  
    .supplyAsync(() -> fetchPayment())  
    .thenApply(payment -> validate(payment))
```

```
.thenApply(payment -> calculateAmount(payment))  
.thenAccept(result -> save(result));
```

This allows asynchronous stages to be composed without manually calling `get()` after every step.

Useful concepts to know:

```
thenApply  
thenAccept  
thenCompose  
exceptionally  
handle  
allOf  
anyOf
```

32. Coordination Utilities

Java's `java.util.concurrent` package provides higher-level synchronization primitives.

The most important ones for interviews are:

```
CountDownLatch  
CyclicBarrier  
Semaphore  
BlockingQueue  
Phaser
```

33. `CountDownLatch`

Use when:

One or more threads need to wait until a set of operations completes.

Example:

```
CountDownLatch latch = new CountDownLatch(3);
```

Three workers:

```
executor.submit(() -> {  
    loadPayments();  
    latch.countDown();  
});  
  
executor.submit(() -> {  
    loadInvoices();  
    latch.countDown();  
});  
  
executor.submit(() -> {  
    loadCustomers();  
    latch.countDown();  
});
```

Main/coordinator:

```
latch.await();

System.out.println("All tasks completed");
```

Conceptually:

```
Task A → countDown()
Task B → countDown()
Task C → countDown()
      ↓
      count = 0
      ↓
    waiting thread
    proceeds
```

Key property

A `CountDownLatch` is generally **one-shot**.

Once it reaches zero, it cannot be reset.

34. `CyclicBarrier`

Used when:

A group of threads must all reach a synchronization point before any continue.

Example:

```
CyclicBarrier barrier =
    new CyclicBarrier(3);
```

Each thread:

```
loadData();

barrier.await();

processCombinedData();
```

Conceptually:

```
Thread A ┌
Thread B ┤ ── Barrier ──> Continue
Thread C └
```

Unlike `CountDownLatch`, a `CyclicBarrier` can be reused for another cycle.

35. Latch vs Barrier

CountDownLatch	CyclicBarrier
One-shot	Reusable
Count decreases	Threads arrive at barrier
One/few threads can await completion	Participant threads wait for each other
Good for "wait until tasks finish"	Good for "wait until everyone reaches this point"

Mental model:

Latch:
"A, B and C finish → then I continue"

Barrier:
"A, B and C all reach here → then everyone continues"

36. Semaphore

A semaphore controls the number of threads that can access a resource concurrently.

Example:

```
Semaphore semaphore = new Semaphore(3);
```

Only 3 permits.

```
semaphore.acquire();

try {
    accessResource();
} finally {
    semaphore.release();
}
```

If 10 threads attempt access:

```
Thread 1  |
Thread 2  |
Thread 3  | → allowed
          |
Thread 4  |
Thread 5  | → waiting
Thread 6  |
...
```

Useful for:

- limiting concurrency
- protecting scarce resources
- connection-like resource limits

37. BlockingQueue

This is one of the most useful practical concurrency abstractions.

Producer:

```
queue.put(payment);
```

Consumer:

```
Payment payment = queue.take();
```

A `BlockingQueue` handles the waiting behavior for you.

Example:

```
BlockingQueue<Payment> queue =  
    new ArrayBlockingQueue<>(100);
```

Producer:

```
queue.put(payment);
```

Consumer:

```
Payment payment = queue.take();  
process(payment);
```

Conceptually:

```
Producer  
  ↓  
BlockingQueue  
  ↓  
Consumer
```

This is generally much cleaner than manually implementing:

```
synchronized  
+  
wait()  
+  
notify()
```

for producer-consumer problems.

38. Phaser

`Phaser` is a more flexible coordination utility for applications where participants can arrive and leave across multiple phases.

You don't need the same depth as `CountDownLatch`, `CyclicBarrier`, or `Semaphore` for most 3-year interviews, but know its purpose.

Think:

```
Phase 1
```

```
↓
all participants synchronize
↓
Phase 2
↓
all participants synchronize
↓
Phase 3
```

39. Important: Lock vs Synchronization vs Coordination

These concepts are related but different.

Lock

Controls:

Who can enter a critical section?

Example:

ReentrantLock

Synchronization / **synchronized**

Provides:

mutual exclusion + visibility

Coordination

Controls:

When should another thread proceed?

Examples:

```
wait/notify
CountDownLatch
CyclicBarrier
Semaphore
BlockingQueue
```

That's a useful distinction in interviews.

40. A Practical Example

Imagine an application processes payments concurrently.

```
100 payment tasks arrive
↓
ExecutorService
↓
Thread Pool
↓
Worker threads
```

Suppose a shared in-memory counter tracks completed payments.

Bad:

```
int completedPayments;

completedPayments++;
```

Possible solution:

```
AtomicInteger completedPayments =
    new AtomicInteger();

completedPayments.incrementAndGet();
```

Suppose processing must use a limited resource:

```
Semaphore semaphore =
    new Semaphore(10);
```

Only 10 workers enter that resource simultaneously.

Suppose startup requires three initialization tasks to finish:

```
Load configuration
Load cache
Load reference data
```

Use:

```
CountDownLatch
```

Suppose workers need to meet at the end of each processing phase:

```
Phase 1
↓
Phase 2
↓
Phase 3
```

Use:

```
CyclicBarrier
```

Suppose you have producer-consumer processing:

```
Payment producer
↓
BlockingQueue
↓
Payment workers
```

Use:

```
BlockingQueue
```

41. Common Interview Trap: `sleep()` Does Not Release Lock

Consider:

```
synchronized void process() throws Exception {  
    Thread.sleep(5000);  
}
```

During those five seconds, the thread still owns the monitor.

So another thread trying to enter the same synchronized method waits.

By contrast:

```
wait();
```

releases the monitor while waiting.

42. Common Interview Trap: `volatile` Does Not Solve Race Conditions

Wrong:

```
volatile int count = 0;  
  
count++;
```

Correct alternatives include:

```
AtomicInteger count = new AtomicInteger();
```

or:

```
synchronized void increment() {  
    count++;  
}
```

depending on the requirements.

43. Common Interview Trap: `synchronized` Does Not Mean “All Threads Are Blocked”

Suppose:

```
synchronized (lockA) {  
    ...  
}
```

Only threads trying to acquire **the same monitor** `lockA` are mutually excluded.

A thread using:

```
synchronized (lockB)
```

can still execute concurrently.

44. Common Interview Trap: Executor Is Not the Same as Thread

An executor manages execution of tasks.

You submit:

```
Runnable  
Callable
```

to it.

The executor decides how worker threads execute those tasks.

Think:

```
Task ≠ Thread  
  
Runnable/Callable → Task  
  
ExecutorService → Task execution management  
  
Thread → Actual execution unit
```

45. Common Interview Trap: `wait()` Must Be Used With the Correct Monitor

This is invalid:

```
public void process() {  
    wait();  
}
```

It throws:

```
IllegalMonitorStateException
```

You need to own the object’s monitor, typically:

```
synchronized
```

Example:

```
synchronized (lock) {  
    lock.wait();  
}
```

Likewise:

```
synchronized (lock) {  
    lock.notifyAll();  
}
```

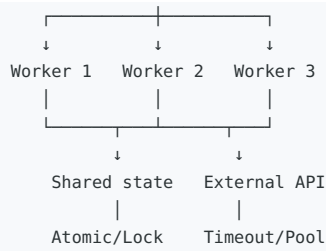
46. Choosing the Right Tool

Requirement	Typical choice
Simple mutual exclusion	synchronized
Advanced lock control	ReentrantLock
Many readers, few writers	ReadWriteLock
Simple visibility flag	volatile
Atomic counter/state update	AtomicInteger , AtomicReference
Manage worker threads	ExecutorService
Get async result	Future
Compose async operations	CompletableFuture
Producer-consumer	BlockingQueue
Wait until N tasks finish	CountDownLatch
Threads meet at synchronization point	CyclicBarrier
Limit concurrent access	Semaphore
Repeated multi-phase coordination	Phaser

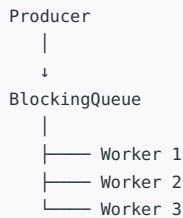
47. High-Level Architecture

A good backend concurrency design might look like:





And if tasks are producer-consumer:



48. Interview-Ready Answer

If an interviewer asks:

“How do you handle multithreading in Java?”

You can answer:

Java provides several mechanisms for concurrency. A thread can be created directly using `Thread`, by implementing `Runnable`, or by using `Callable` with an executor. In production applications I would generally prefer `ExecutorService` because it provides thread-pool management and controls concurrency. For shared mutable state, `synchronized`, `Lock`, atomic classes, and `volatile` can be used depending on whether the requirement is mutual exclusion, atomicity, or visibility. For thread coordination, Java provides `wait/notify`, but higher-level utilities such as `BlockingQueue`, `CountDownLatch`, `CyclicBarrier`, and `Semaphore` are generally easier to reason about. For asynchronous workflows, `Future` and `CompletableFuture` are also useful.

49. The Most Important Distinctions to Memorize

```

volatile
  → visibility

AtomicInteger
  → atomic operations

synchronized
  → mutual exclusion + visibility

ReentrantLock
  → mutual exclusion + advanced lock control
  
```

```
ExecutorService
    → manage task execution / thread pools

Future
    → represent async result

CompletableFuture
    → compose async operations

wait/notify
    → low-level thread coordination

BlockingQueue
    → producer-consumer coordination

CountDownLatch
    → wait until N operations complete

CyclicBarrier
    → wait until participants reach a point

Semaphore
    → limit concurrent access
```

50. BPS-Specific Questions — Inspect Your Codebase

Don't assume these are implemented in BPS; inspect the code and identify what is actually present.

Thread creation

- Does BPS explicitly create `Thread` objects?
- Is `ExecutorService`, `@Async`, scheduled execution, or another framework mechanism used?
- Where are asynchronous tasks created?

Shared state

- Is there any mutable static/in-memory state shared between requests?
- Do you find `volatile`, `AtomicInteger`, `AtomicReference`, `synchronized`, or `ReentrantLock`?

Payment processing

- Does recurring-payment batch processing use multiple worker threads or sequential processing?
- If multiple payments execute concurrently, what prevents duplicate processing?
- Is concurrency controlled at the JVM level, database level, or by the external payment provider?

Coordination

- Are there producer-consumer patterns?
- Is `BlockingQueue` used?
- Are there latches, semaphores, futures, or `CompletableFuture`?

External calls

- When BPS calls Hydra/Chase, are calls synchronous or executed concurrently?
- Is there an executor/thread pool controlling outbound concurrency?
- What happens when all worker threads are occupied?

Very important interview question

“Suppose 100 payment requests arrive simultaneously. How would you control concurrency so that you don’t create 100 uncontrolled threads or overload Hydra/Chase?”

A strong answer should lead you toward:

```
bounded ExecutorService
+
connection pool limits
+
timeouts
+
rate limiting
+
circuit breaker
+
idempotency
```

That connects this topic directly to the **Spring Boot microservices/concurrency architecture** you have already been studying.